

技术文档

端侧选择

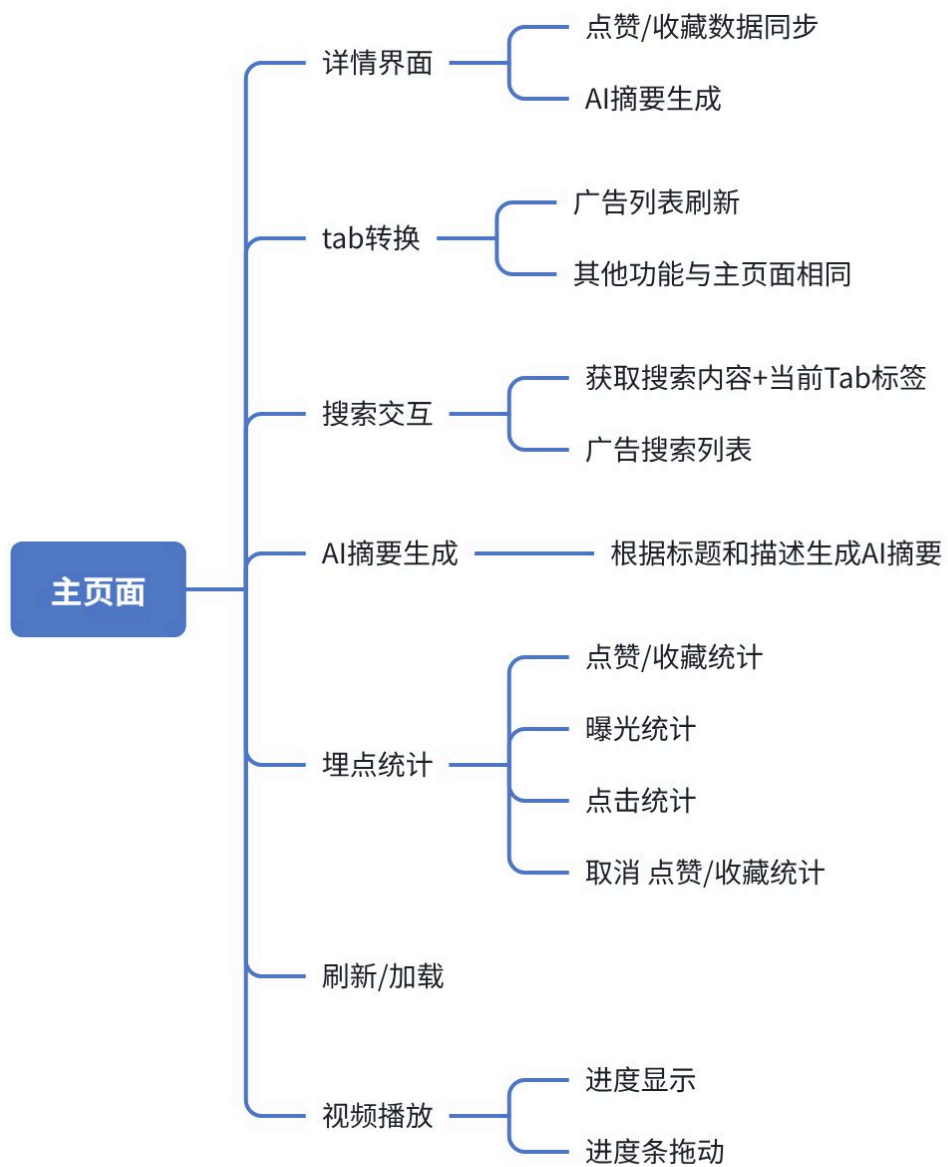
Android

运行环境

模拟器Medium Phone



整体功能框架



技术模块

模块	职责描述
MainActivity	主界面，管理广告列表展示和 Tab 切换
AdDetailActivity	广告详情页，展示视频播放和互动功能
StatisticsActivity	统计页面，展示埋点数据汇总
AdAdapter	RecyclerView 适配器，处理广告卡片渲染和视频播放
AnalyticsManager	埋点统计管理器，负责事件追踪和数据持久化
PlayerPool	视频播放器池，管理 MediaPlayer 复用
MockData	模拟数据生成器
DeepSeekApiManager	DeepSeek API 调用管理器

具体模块介绍

加载（recyclerView.addOnScrollListener）

1. 设置每页广告数，到达页底部，进行加载
2. 目前页面见广告数（int visibleItemCount = layoutManager.getChildCount()）
3. 返回当前屏幕中完全或部分可见的第一个条目在适配器中的位置索引（int firstVisibleItemPosition = layoutManager.findFirstVisibleItemPosition();）
4. 适配器中的数据数量（int firstVisibleItemPosition = layoutManager.findFirstVisibleItemPosition();）
5. 如果可见数+可见的第一条>=总数量，加载更多数据

加载更多数据

1. 当前页加1
2. 得到新页广告列表，如果列表大于0，进行3；否则输出提示没有更多了
3. 原始广告列表中添加新页广告列表
4. 通知 RecyclerView 的适配器，在指定位置范围内插入了新数据项，以便列表仅刷新这部分区域，而不是整个列表（adAdapter.notifyItemRangeInserted()）

刷新（`swipeRefresh.setOnRefreshListener(...)`用于监听用户的下拉刷新动作）

1. 进行刷新转圈动作（`swipeRefresh.setRefreshing(true);`）
2. 清空Adlist，当前Tab广告顺序重新随机排列添加到Adlist中（Mockdata）
3. 通知列表：整个数据集已经发生变化，请重新刷新所有可见条目（`adAdapter.notifyDataSetChanged();`）
4. 停止刷新动画，并将控件重置为“非刷新中”的状态（`swipeRefresh.setRefreshing(false);`）

点击到详情页

1. 设置广告监听，记录广告位置用于返回（`adAdapter.setOnAdClickListener`）
2. 点击次数埋点统计：（`analyticsManager.trackClick(ad, currentCategory);`）
3. 跳转到详情页，并且广告携带数据
4. 设置跳转动画：`overridePendingTransition(R.anim.slide_in_right, R.anim.slide_out_left)`

详情页

1. 加载控件
2. 使用广告数据初始化详情页各个组件（文本组件`id.setText()`；视频组件`FrameLayout videoContainer.setVisibility(View.VISIBLE)`；图像组件`ivAd.setVisibility(View.VISIBLE)`；点赞/收藏组件）
3. 设置点击监听：点赞，收藏，分享（显示一个短暂的提示消息（`Toast`））
4. 设置返回监听，返回数据（`setResult(RESULT_CODE_UPDATE, intent);`）

返回主页面（`onActivityResult`）

1. 得到相应广告，主页面广告（`adList.get(clickedPosition)`），返回的数据（`data.getSerializableExtra`）
2. 更新主页面广告的点赞和收藏
3. 通知 `RecyclerView`（或 `ListView`）的适配器数据已变化，刷新整个列表界面（`adAdapter.notifyDataSetChanged();`）

Tab选择监听（（`tabLayout.addTabSelectedListener`））

tab选择(`onTabSelected`)

1. 得到新tab标签文本 (tab.getText().toString())
2. 刷新
3. 回到顶部(recyclerView.scrollToPosition(0);)

Tab回调：onTabReselected用于搜索时当用户再次点击当前已经选中的 Tab(同上)

搜索按钮监听

1. 得到搜索框内容：etSearch.getText().toString().trim();
2. 得到广告标签属性中包含tab标签的广告
3. 找到广告描述，标题，摘要，标签中包含包含搜索内容的广告
4. 清空广告列表，添加新广告
5. 通知 `RecyclerView` (或 `ListView`) 的适配器数据已变化，刷新整个列表界面
(adAdapter.notifyDataSetChanged();)

```
⚽ boolean keywordMatches = ad.getTitle().toLowerCase().contains(query)
    || ad.getDescription().toLowerCase().contains(query)
    || ad.getAiSummary().toLowerCase().contains(query)
    || ad.getSource().toLowerCase().contains(query);
for (String tag : ad.getTags()) {
    if (tag.toLowerCase().contains(query)) {
        keywordMatches = true;
        break;
    }
}
```

点赞/收藏

1. 广告数据的isLiked/isFavorited初始为false
2. 点击监听 (adAdapter.setOnAdClickListener((ad, position))

3. 对广告点赞/收藏的数据isLiked/isFavorited取反
4. 如果isLiked/isFavorited为真，则使用用红色爱心和黄色收藏图标填充imageView;为假使用灰色填充



点赞

```
adHolder.ivLike.setOnClickListener(v -> {  
    boolean wasLiked = ad.isLiked();  
    ad.setLiked(!ad.isLiked());  
    ad.setLikes(ad.isLiked() ? ad.getLikes() + 1 : ad.getLikes() - 1);  
    adHolder.ivLike.setImageResource(ad.isLiked() ? R.drawable.ic_liked :  
R.drawable.ic_like);  
    adHolder.tvLikes.setText(String.valueOf(ad.getLikes()));  
....})
```

收藏

```
adHolder.ivFavorite.setOnClickListener(v -> {  
    boolean wasFavorited = ad.isFavorited();  
    ad.setFavorited(!ad.isFavorited());  
    ad.setFavorites(ad.isFavorited() ? ad.getFavorites() + 1 : ad.getFavorites() - 1);  
    adHolder.ivFavorite.setImageResource(ad.isFavorited() ? R.drawable.ic_favorited :  
R.drawable.ic_favorite);  
    adHolder.tvFavorites.setText(String.valueOf(ad.getFavorites()));  
....})
```

AI大模型调用

1. 列表项需要显示时，填充数据onBindViewHolder
2. 得到广告的标题与描述 (DeepSeekApiManager.getInstance().generateSummary(ad.getTitle(), ad.getDescription()))



DeepSeekApiManager

1. 编写prompt
2. 调用API模型：callDeepSeekApi(String prompt) throws IOException, JSONException，返回摘要字符串

3. 调用监听：DeepSeekApiManager.OnSummaryGeneratedListener(), 返回 listener.onSuccess(result.trim());
4. 成功（onsuccess）则设置广告摘要，在对应组件显示摘要文本

视频播放

1. 在adAdapter.onBindViewHolder中判断广告类型是否是视频，如果则进行下一步
2. 获取视频的第一帧作为视频封面



```
android.net.Uri uri = android.net.Uri.parse(videoUrl);
```

```
Glide.with(context)
    .load(uri)
    .placeholder(R.drawable.ic_image_placeholder)
    .error(R.drawable.ic_image_error)
    .apply(new RequestOptions()
        .centerCrop())
    .into(imageView);
```

3. 显示视频时长



```
holder.tvDuration.setText(formatDuration(ad.getDuration()));
```

```
private String formatDuration(int seconds) {
    int mins = seconds / 60;
    int secs = seconds % 60;
    return String.format("%02d:%02d", mins, secs);
}
```

4. 设置视频区域点击监听（holder.mediaContainer.setOnClickListener）
5. 如果有其他视频在播放，先停止视频播放并清理与之关联的进度更新任务，停止进度更新，释放资源



```
if (holder.mediaPlayer != null && holder.mediaPlayer.isPlaying()) {
    holder.mediaPlayer.pause();
}
```

```
private void stopProgressUpdate(AdViewHolder holder){
    if (holder.progressRunnable != null) {
        holder.progressHandler.removeCallbacks(holder.progressRunnable);
        holder.progressRunnable = null;
    }
    playerPool.releasePlayer(playerId);
}
```

6. 将 `SurfaceView` 设为可见，从 `SurfaceView` 中获取 `SurfaceHolder`，后续通过它设置视频画面的显示位置
7. 设置循环播放 (`player.setLooping(true);`)
8. 将解析后的 视频 `Uri` 设置给 `MediaPlayer`，指定要播放的视频
(`player.setDataSource(context, android.net.Uri.parse(videoUrl));`)
9. 异步加载视频，避免阻塞主线程 (`player.prepareAsync();`)
10. 设置准备完成监听器，确保 `Surface` 已经创建且有效，再将视频画面输出到 `SurfaceView` 上;设置总时长和进度条



```
player.setOnPreparedListener(mp -> {
    // 准备完成后再设置display，确保surface已就绪
    if (holderSurface.getSurface().isValid()) {
        mp.setDisplay(holderSurface);
    }

    mp.start();
    holder.isPlaying = true;

    holder.tvDuration.setText(formatTime(mp.getDuration()));
    holder.seekBar.setMax(mp.getDuration());
    holder.seekBar.setProgress(0);
    holder.tvCurrentTime.setText("00:00");
    setupSeekBarListener(holder);
    startProgressUpdate(holder);

    // 隐藏封面，显示视频控件
    holder.ivVideoCover.setVisibility(View.GONE);
    holder.ivPlay.setVisibility(View.GONE);
    holder.videoControlBar.setVisibility(View.VISIBLE);
}
```



```
});
```

11. 设置进度条的拖动监听，改变进度
12. 实现进度条拖动，进度条的实时更新
13. 播放完成监听器（`player.setOnCompletionListene`）



```
mp.seekTo(0);
```

将播放器的进度跳转到开头（0 毫秒位置）。

```
holder.seekBar.setProgress(0);
```

将进度条（SeekBar）的进度重置为 0，表示回到起点。

```
holder.tvCurrentTime.setText("00:00");
```

将当前播放时间的 TextView 显示为 "00:00"，与进度条同步。

埋点统计

曝光统计

1. 个广告卡片进入屏幕可见区域时，立即上报一次“曝光”事件（`onViewAttachedToWindow` 是 RecyclerView 的 Adapter 回调，当某个 ViewHolder 所对应的视图被附着到窗口（即实际显示在屏幕上）时调用）
2. 生成一个完整、可追溯的埋点事件，包含了广告 ID、什么事件（曝光/点击）、何时（时间戳）、在哪里（分类）以及唯一的追踪 ID

```
private AnalyticsEvent createEvent(Ad ad, String eventType, String category) {  
    String eventId = UUID.randomUUID().toString();  
    long timestamp = System.currentTimeMillis();  
    return new AnalyticsEvent(eventId, ad.getId(), ad.getTitle(), eventType, timestamp,  
        category);  
}
```

3. 向事件列表添加曝光事件
4. 广告的曝光次数+1

5. 保存事件



保存事件

1. 获取 `SharedPreferences` 的编辑器对象，用于执行写入操作
(`SharedPreferences.Editor editor = sharedPreferences.edit();`)
2. 创建一个 `StringBuilder`，用于高效拼接字符串 (`StringBuilder sb = new StringBuilder();`)
3. 将每个事件的各个字段按固定顺序拼接，记录曝光广告的标题，描述，摘要，标签等
4. 将整个拼接好的字符串存入 `SharedPreferences`，键名为 `KEY_EVENTS`
(`editor.putString(KEY_EVENTS, sb.toString());`)
5. 异步提交修改，不会阻塞当前线程 (`editor.apply();`)

点击事件

1. 点击监听 (`adAdapter.setOnAdClickListener((ad, position))`)
2. 生成一个完整、可追溯的埋点事件，事件类型 `AnalyticsEvent.TYPE_CLICK`
3. 保存事件

点赞/收藏事件

1. 点击监听 (`adHolder.ivLike.setOnClickListener; adHolder.ivFavorite.setOnClickListener;`)



点赞/取消点赞埋点

```
if (ad.isLiked()) {  
    analyticsManager.trackLike(ad, currentCategory);  
} else {  
    analyticsManager.trackUnlike(ad, currentCategory);  
}
```

收藏/取消收藏埋点

```
if (ad.isFavorited()) {  
    analyticsManager.trackFavorite(ad, currentCategory);  
}
```

```
} else {  
    analyticsManager.trackUnfavorite(ad, currentCategory);  
}
```

2. 生成一个完整、可追溯的埋点事件，事件类型

(AnalyticsEvent.TYPE_UNFAVORITE/AnalyticsEvent.TYPE_FAVORITE/
AnalyticsEvent.TYPE_UNLIKE/AnalyticsEvent.TYPE_LIKE)

3. 保存事件

遇到问题及解决

视频实现手动拖动改变进度

1. 对进度条设置点击监听 (holder.seekBar.setOnSeekBarChangeListener)
2. 当进度条的进度发生变化时被调用onProgressChanged(SeekBar seekBar, int progress, boolean fromUser)
3. 只有当 用户主动拖动 并且 播放器对象存在 时，才执行内部逻辑
4. 调用 MediaPlayer 的 seekTo 方法，将播放位置跳转到用户拖动的进度值 (holder.mediaPlayer.seekTo(progress);)
5. 更新显示当前播放时间的 TextView (holder.tvCurrentTime.setText(formatTime(progress));)

视频进度条与时间的实时更新

1. 在确保surface已就绪后 (player.setOnPreparedListener)
2. 启动周期性进度刷新 (startProgressUpdate)
3. 创建一个新的 Runnable 对象，赋值给 holder.progressRunnable。
Runnable 内部定义了需要周期性执行的代码块
4. 获取当前播放位置 currentPos (int currentPos = holder.mediaPlayer.getCurrentPosition();)
5. 将 currentPos 格式化成 mm:ss 并显示到 tvCurrentTime 上
(holder.tvCurrentTime.setText(formatTime(currentPos));)
6. 将进度条的当前进度设置为 currentPos (holder.seekBar.setProgress(currentPos);)
- 使用 progressHandler 在 1000 毫秒 (1 秒) 后再次执行当前的 Runnable (即 this)，从而实现周期性循环执行。(holder.progressHandler.postDelayed(this, 1000);)

视频播放时与广告不匹配

原因

当旧的视频移出画面后，播放器没有释放

解决方法

1. 在列表项被移出屏幕或不再使用时，及时释放与之关联的视频播放器资源（`public void onViewRecycled(@NonNull RecyclerView.ViewHolder holder)`）
2. 获取播放器ID,如果 `playerId` 存在且播放器池 `playerPool` 不为空，则调用 `releasePlayer(playerId)` 释放该播放器。